
DARASA-CORE: THE MASTER ARCHITECTURAL BLUEPRINT (v4.0)

CLASSIFICATION: STRICTLY CONFIDENTIAL / ENGINEERING EYES ONLY

TARGET DOMAIN: KENYAN SENIOR SECONDARY (CBC FRAMEWORK)

ARCHITECTURE: EVENT-DRIVEN MODULAR MONOLITH ("THE CITADEL")

PART 1: THE PROBLEM-SOLUTION MATRIX

Features fail if they do not map to reality. Darasa-Core solves the four fundamental CBC crises through applied computer science:

- The Parent Problem ("We want Top 10s"):** Parents do not understand raw CBC rubric scores.
 - Algorithmic Solution: Rule-Based NLP & Vector Mapping.* Darasa translates scalar arrays (e.g., [3, 4, 2]) into human-readable Abstract Syntax Trees (AST), generating localized, empathetic text summaries without the ruinous API costs of LLMs.
- The Materials Nightmare ("Kids forget practical requirements"):**
 - Algorithmic Solution: Distributed Temporal Event Bus.* Using Redis Sorted Sets and Celery ETA tasks, the system schedules precise, localized SMS alerts to parents 72 hours before a practical, executing in $O(1)$ time without database polling.
- The Compiling Headache ("Portals crash under load"):**
 - Algorithmic Solution: Optimistic Concurrency Control (OCC).* The Vue 3 Fast-Grid operates offline-first, processing math in the browser. It sends a single, compressed JSON payload to the Django backend using `bulk_update()`, preventing database deadlock.
- The Scheme of Work Burden ("Endless repetitive planning"):**
 - Algorithmic Solution: PostgreSQL Recursive CTEs.* Darasa uses deep Directed Acyclic Graph (DAG) traversal at the database metal to clone hundreds of relational syllabus rows in a single atomic $O(1)$ execution plan.

PART 2: ARCHITECTURE & EVENT-DRIVEN TOPOLOGY

Darasa-Core strictly adheres to the following principles to ensure scalability and legal compliance:

- **Domain-Driven Design (DDD):** The codebase is strictly partitioned into business capabilities (e.g., grading, academics). Cross-domain synchronous database joins are physically banned.
- **Command Query Responsibility Segregation (CQRS):** Write operations (teachers submitting grades) are separated from Read operations (principals viewing analytics) at the database level to prevent locking under heavy end-of-term loads.
- **The Transactional Outbox Pattern:** To guarantee Eventual Consistency, database writes and event emissions occur simultaneously without two-phase commit locking. A synchronous transaction commits the primary data and an **OutboxEvent** record simultaneously. A background Polling Relay fetches events in batches and dispatches them to the Redis message broker.

PART 3: CONQUERING THE 8 FALLACIES OF DISTRIBUTED COMPUTING

Network volatility is inevitable. Darasa mitigates these fallacies natively:

- **"The Network is Reliable" (Strict Idempotency):** Networks drop packets and retry. Every Celery event is assigned a unique **Event_UUID**. The API checks a high-speed Redis cache using a **SETNX** lock; if the UUID exists, the system returns a 200 OK but drops the duplicate payload.
- **"Latency is Zero" (Asynchronous Offloading):** The API immediately responds to heavy compilation requests with **202 Accepted** and offloads the work to Celery queues.
- **"Bandwidth is Infinite" (BFF Payload Compression):** Backends for Frontends (BFFs) aggregate data and transmit mathematically compressed JSON to the client, minimizing payload weight for rural Kenyan schools.
- **"The Network is Secure" (Zero-Trust Event Bus):** All JSON payloads pushed to Celery/Redis are cryptographically signed using a server-side HMAC-SHA256 key. Consumers verify the signature before execution.
- **"Transport Cost is Zero" (Claim-Check Pattern):** Passing 10MB lesson plans through Django consumes massive RAM. Darasa generates presigned URLs, forcing clients to upload files directly to Cloudflare R2.

PART 4: DATA STRUCTURES & DATABASE PHYSICS

We implement raw mathematical structures to ensure predictable performance at scale.

- **Database Sharding & Tenant Routing (Hash Tables):** A single PostgreSQL instance cannot handle 1,000 schools. We abandon the naive modulo hash and map the hash space to a Consistent Hashing Ring. If a database node fails, only the tenants cryptographically mapped to that specific node are re-routed, ensuring 66% of the hardware experiences zero disruption.
- **CBC Curriculum Prerequisite Mapping (Graph Theory):** The curriculum domain maps the entire KNEC syllabus as a Directed Acyclic Graph (DAG). The Darasa

Compiler uses a Depth-First Search (DFS) topological sort across a student's scores to pinpoint the exact foundational vertex where understanding collapsed.

- **The RBAC Authorization Engine (Tries):** The API Gateway holds a Prefix Tree (Trie) of all secured endpoints in memory. The authorization check drops to $O(\log n)$ memory traversal, allowing the gateway to reject unauthorized requests before waking a Django thread.
- **Temporal Client State (Doubly Linked Lists):** The Vue 3 client maintains a localized Doubly Linked List of the last 15 JSON state changes. Pressing 'Undo' traverses the `prev` pointer in strictly $O(1)$ time with zero network calls.

To fully understand how **Optimistic Concurrency Control (OCC)** solves the teacher's compiling headache, interact with this Fast-Grid simulator:

Show me the visualisation

PART 5: SECURITY & DPA LEGAL ARMOR

Darasa processes the sensitive data of minors. Compliance is a hard architectural requirement.

- **Absolute Multi-Tenancy:** PostgreSQL uses Schema-per-Tenant isolation (`django-tenants`). A global connection variable is set per request, and PostgreSQL Row-Level Security (RLS) physically rejects queries trying to read schema data outside the bound tenant.
- **End-to-End Data Protection:** Minor PII (e.g., medical records) is protected via Field-Level Encryption using `cryptography.fernet` at the Django ORM layer before writing to disk.
- **The Access Panopticon:** The system records every `GET` request directed at a student's profile into an append-only `AccessAuditLog`, fulfilling the Data Protection Act's transparency and accountability mandates.
- **Right to be Forgotten:** Background Celery cron jobs execute nightly. Soft-deleted profiles are subjected to cryptographic shredding after 30 days to comply with legal mandates.

PART 6: CHAOS ENGINEERING & TDD PROTOCOLS

We actively anticipate and survive hardware destruction.

- **Crash-Only Software:** Celery workers are configured to intentionally terminate and reboot (e.g., `max_tasks_per_child = 1000`) to prevent memory leaks, relying on Docker's orchestration to instantly spin up replacements.
- **Design by Contract (DbC):** Core engines enforce strict pre/post-conditions (e.g., `assert 1 <= rubric_score <= 4`). Violations trigger a Fail-Stop, halting the transaction to prevent database corruption.
- **Fault Injection (Toxiproxy):** During CI/CD, the Toxiproxy service intercepts the testing database connection and injects 1000ms latency and 5% packet loss. The test suite must pass under these degraded conditions.

- **The Rubric Fuzz Test:** We use the `Hypothesis` library to fire thousands of edge-case inputs (null characters, huge floats, SQL injections) at the Fast-Grid endpoint to guarantee the API safely rejects anomalies.

PART 7: OBSERVABILITY & TELEMETRY

Observability mathematically proves the DSA implementations function in production.

- **Distributed Tracing:** We implement OpenTelemetry to inject a cryptographic `Trace_ID` at the gateway, tracking execution cost across every micro-component.
- **The Big O Enforcer ($O(n^2)$ Regression Detector):** The Datadog/Sentry APM actively monitors SQL query counts per HTTP cycle. If the engine detects >15 identical `SELECT` statements in a single trace (The N+1 Query Problem), it triggers a catastrophic slack alert. The fix mandates SQL `JOIN` operations to restore query complexity to $O(1)$.
- **The Circuit Breaker Proxy:** If an external dependency (like SMS) fails, the system tracks consecutive timeouts. If they exceed 5, it transitions to an `OPEN (Fail-Fast)` state, aborting outbound connections instantly and routing payloads to an internal fallback table.
- **Asynchronous Queue Analytics:** We continuously monitor the length of Redis lists. If the compiler queue depth exceeds 500 items, the Kubernetes autoscaler triggers, provisioning additional stateless Celery pods to drain the queue.

This is the system. It is legally compliant, algorithmically sound, and directly addresses the pain of the users on the ground.

PART 4: DATA STRUCTURES & DATABASE PHYSICS (EXPANDED)

We must define exactly how the algorithms interact with the PostgreSQL disk and the server RAM.

4.1 Consistent Hashing & Virtual Nodes (Database Sharding)

- **The Deep Context:** A standard modulo hash distributes tenants unevenly. If `School_A` has 5,000 students and `School_B` has 100, standard hashing might place both on the same physical server, causing a CPU bottleneck.
- **The Low-Level Implementation:** We implement **Virtual Nodes (vnodes)**. Instead of hashing `Database_Node_1` once on the ring, we hash it 100 times using cryptographic algorithms (e.g., `hashlib.md5(f"Node_1_vnode_{i}").hexdigest()`).
- **The Execution:** When a massive National School is onboarded, its hashed domain intersects with the ring. Because the physical databases are distributed as hundreds of virtual nodes, the load of massive schools is mathematically guaranteed to be

distributed evenly across your entire physical server rack, preventing localized RAM exhaustion.

4.2 Directed Acyclic Graph (DAG) Traversal via Recursive CTEs

- **The Deep Context:** Copying a Scheme of Work involves duplicating a tree structure: `Term` $\rightarrow$ `Weeks` $\rightarrow$ `Lessons`. Doing this in Python requires thousands of `INSERT` statements, locking the database thread and causing the API to timeout.
- **The Low-Level Implementation:** We bypass the Django ORM entirely for this specific operation. We execute a raw PostgreSQL `WITH RECURSIVE` Common Table Expression.
- **The Execution:** The database engine itself traverses the DAG in memory. It generates new UUIDv4 primary keys for the cloned rows while instantly mapping the new Foreign Key relationships to match the old hierarchy. A process that takes 30 seconds in Python takes 40 milliseconds at the database metal.

PART 5: SECURITY & DPA LEGAL ARMOR (EXPANDED)

The Kenya Data Protection Act (2019) and the Children's Act mandate that exposing a minor's Personally Identifiable Information (PII) is a criminal offense. Perimeter security (firewalls) is not enough; we must assume the database will eventually be breached.

5.1 Application-Layer Field-Level Encryption (FLE)

- **The Deep Context:** If a hacker dumps the `darasa_citadel` PostgreSQL volume, or if a rogue server admin looks at the tables, they must not be able to read medical records, disciplinary notes, or National IDs.
- **The Low-Level Implementation:** We utilize a custom `EncryptCharField` in Django, powered by the `cryptography.fernet` library.
- **The Execution:** Encryption and decryption happen strictly in the server's RAM, *never* in the database. When a teacher submits a medical note, Django uses a master Key Management Service (KMS) key to symmetrically encrypt the string. The database only receives and stores the ciphertext (e.g., `gAAAAABk1...`).

To understand the sheer power of this legal armor, interact with this visualizer:

Show me the visualisation

5.2 The High-Velocity Access Panopticon (Audit Logging)

- **The Deep Context:** The ODPC (Office of the Data Protection Commissioner) requires a pristine audit trail of exactly who viewed a child's record. Writing an audit log to PostgreSQL for every single HTTP `GET` request will crash your primary database.
- **The Low-Level Implementation:** We decouple the audit logs from the transactional database. We write a custom Django Middleware that intercepts every request.

- **The Execution:** The middleware extracts the `Actor_UUID` (from the JWT) and the `Target_UUID` (from the URL parameter). It fires a non-blocking, asynchronous UDP packet containing this data to a specialized Time-Series Database (like ClickHouse or an ELK stack). This guarantees absolute legal traceability without adding a single millisecond of latency to the teacher's UI.

PART 6: CHAOS ENGINEERING & TDD PROTOCOLS (EXPANDED)

Standard unit tests only verify that the code works under perfect conditions. Chaos engineering verifies how the code behaves when the infrastructure is actively bleeding.

6.1 Mutation Testing (`mutmut`)

- **The Deep Context:** A developer writes a test for the grading compiler, and the test passes. But is the test actually checking the math, or is it a false positive?
- **The Low-Level Implementation:** During the GitHub Actions CI/CD pipeline, the `mutmut` library dynamically alters the Abstract Syntax Tree (AST) of your Python source code. It changes `if score >= 3:` to `if score > 3:`.
- **The Execution:** It then runs your `pytest` suite. If your test suite still says "Passed" despite the core logic being sabotaged, the CI/CD pipeline fails the build. Your tests must be brittle enough to break when the code is wrong.

6.2 Network Sabotage (`Toxiproxy`)

- **The Deep Context:** The API works perfectly on a local Docker setup. But in production, the connection between the Django container and the Redis container will inevitably stutter.
- **The Low-Level Implementation:** `Toxiproxy` acts as a malicious proxy router sitting between your microservices in the test environment.
- **The Execution:** Before the automated test fires a massive "Compile Term Grades" payload, `Toxiproxy` restricts the Redis TCP bandwidth to 10kbps and injects a 2000ms latency jitter. The test asserts that the Django `teacher-bff` correctly triggers the Circuit Breaker, caches the payload safely in the PostgreSQL Outbox, and returns a graceful `202 Accepted` to the client instead of hanging the HTTP thread and crashing.

PART 7: OBSERVABILITY & TELEMETRY (EXPANDED)

If a principal calls to complain that the system is "slow," you cannot fix "slow." You need mathematical proof of exactly which function call is burning CPU cycles.

7.1 OpenTelemetry & Distributed Context Propagation

- **The Deep Context:** A single click on the Vue 3 frontend triggers a cascade: API Gateway $\rightarrow$ Django BFF $\rightarrow$ Celery Worker $\rightarrow$ PostgreSQL $\rightarrow$ Redis.

- **The Low-Level Implementation:** We utilize the W3C Trace Context standard. The NGINX gateway generates a unique 16-byte `Trace_ID` and injects it into the HTTP header `traceparent`.
- **The Execution:** Every microservice that touches this request reads the `Trace_ID` and appends its own execution time (a `Span`) to the log. You gain a visual waterfall graph showing exactly how many milliseconds the network transport took versus the actual database query.

7.2 The $O(n^2)$ APM Regression Detector

- **The Deep Context:** A Junior Developer builds a feature to list all students in a class. In the Django template/serializer, they call `student.disciplinary_records.all()`. This triggers the $N+1$ query problem: 1 query to fetch the students, and 50 individual queries to fetch the records for each student.
- **The Low-Level Implementation:** Application Performance Monitoring (APM) tools (Datadog/Sentry) natively intercept the Django database cursor.
- **The Execution:** We configure a strict APM threshold. If the tracer detects that the exact same parameterized SQL statement is executed in a loop more than 10 times within a single `Trace_ID`, it flags an $O(n^2)$ Database Regression. An automated alert is fired to the engineering Slack channel, forcing the developer to rewrite the endpoint using `prefetch_related()` before the system scales and collapses.